

that messages are never lost, nor do servers crash during an election, a leader is elected when all other servers are its followers. The code is shown in Figure 5.22 (again, omitting many details).

#### 5.4.4 Example: Leader election in Raft

As another example of a leader-election algorithm that is in practical use, let us take a look at **Raft** [Ongaro and Ousterhout, 2014]. We will return to Raft in Chapter 8 when discussing consensus algorithms and focus here only on how it selects a leader.

Raft operates in a setting in which a handful of known replicated servers (typically five) collaborate by ensuring that each server executes the same set of operations, and all in the same order. To this end, one of the servers is elected as leader to tell the others what the exact order is of those operations. The protocol assumes that messages may be lost and that servers may crash.

Each server can be in one of three states: follower, candidate, or leader. Furthermore, the protocol operates in **terms**, where during each term there is exactly one leader, although it could have perhaps crashed. Each term is numbered, starting with 0. The leader is assumed to regularly send out a message, either containing information on an operation that should be carried out, or otherwise a heartbeat message to tell the other servers that their leader is still up and running.

Each server initially starts in the follower state (that is, there is initially no leader). Adopting the terminology from Howard et al. [2015], after a *follower timeout*, a following server concludes that the leader may have crashed (which, by the way, may be a false conclusion). As a result, it enters the candidate state and starts an election, volunteering to be the new leader. An election starts with a broadcast to all other servers, along with increasing the term number by 1. At that point, three situations may happen.

1. A candidate may receive a message from an alleged leader. If that server indicates it is operating in the same term as the candidate server, the latter will become follower again for the current term.
2. When a following server receives an election message for the first time (in a new term), it simply votes for the candidate and ignores any other election messages (for that new term). Therefore, a candidate server can also receive a vote. If it has a majority of votes (i.e., more than half of the servers, including itself), it promotes itself as leader for the new term.
3. As long as there is no alleged leader, or not enough votes have been received, the candidate server waits until a *candidate timeout* happens. At that point, the candidate server will simply start a new election (and again, for a next term).

Of course, if all servers enter the candidate state, we bear the risk of indefinitely not being able to cast enough votes for a single server. A simple solution is to slightly vary the *follower timeout* and *candidate timeout* on a per-server basis. The result is that, generally, there is only a single candidate for the new term, allowing each other server to vote for that candidate and become a follower. As soon as a server becomes leader, it will send out a heartbeat message to the rest.

### 5.4.5 Elections in large-scale systems

Many leader-election algorithms apply to only relatively small distributed systems. In such cases, the various nodes exchange information among each other, often through several rounds, to eventually collectively decide on the new leader. In Chapter 8, we shall see that this collective decision-making can take place even when a fraction of the participating nodes exhibits nonconforming behavior, as in the case of security attacks or faults. The problem of electing a leader becomes rather nasty when there are potentially many processes to choose from. This is the case, for example, in **permissionless blockchains**.

#### Proof of work

A relatively simple, yet by now heavily criticized solution, is to have the candidates run a computational race, referred to as **proof of work**. Running such a race, which is done by solving a computational puzzle, should be known to be possible, yet difficult. The first process solving the puzzle, wins, and may proceed as leader to append a block of validated transactions to the existing blockchain.

The type of problem used for blockchains is based on what is known as **hashing**. As introduced in Section 1.2.5, a hash function takes as input a (possibly large) data set and produces a fixed-length string of bits, typically of length 1024 or 2048, called a *hash*. A cryptographically well-defined hash function has several important properties [Ferguson and Schneier, 2003]:

1. Computing the hash of a given data set is relatively easy, i.e., it does not require significant computational efforts.
2. However, given a specific hash, it is computationally very difficult to find a corresponding data set with the same associated hash.
3. With very high probability, any two different input data sets will lead to two seemingly unrelated, different hashes. Even if the two data sets are minimally different, their associated hashes will most likely be very different.